ARBITRAGE FINDER

Code Explained

A practical guide to the software delivered in this repository.

Edition: 2 November 2026 · English

Source: docs/code-explained.md
Content fingerprint: f00c5cb82c8c

Read the support status and limitations before using live odds. A working demo and parser tests do not establish that a bookmaker feed is currently accessible.

Contents

1. How to read this guide . . . 4

2. The architecture in one pass . . . 4

2.1 Why Python and a small browser frontend . . . 4

2.2 Main files . . . 4

3. Startup and application construction . . . 5

4. The canonical Quote model . . . 6

4.1 Event fields . . . 6

4.2 Market fields . . . 6

4.3 Canonical spread orientation . . . 6

4.4 Quote and provenance fields . . . 6

5. Settings and their units . . . 7

6. Connector interfaces . . . 7

6.1 Why there are no guessed bookmaker selectors . . . 8

7. Public network reads . . . 8

7.1 Robots and denial behavior . . . 8

7.2 Anonymous browser mode . . . 8

8. HTML extraction . . . 9

9. JSON extraction and transforms . . . 9

10. The optional provider API . . . 10

10.1 Quota handling . . . 10

11. Freshness and quote filtering . . . 10

12. Event and settlement matching . . . 11

12.1 Latest quote and conflicts . . . 11

13. Arbitrage math and stake allocation . . . 11
13.1 Decimal arithmetic . . . 12
13.2 The rounded plan . . . 12
13.3 Boundaries of this model . . . 12
14. Manual scan orchestration . . . 12
15. HTTP API and request guards . . . 12
16. SQLite and credential encryption . . . 13
17. Frontend behavior . . . 14
18. Tests and what they prove . . . 14
19. A safe way to extend the program . . . 14
19.1 Add a dedicated connector . . . 14
19.2 Add a market type . . . 14
19.3 Add event aliases . . . 15
19.4 Add fees or balances . . . 15
20. PDF generation and documentation maintenance . . . 15
21. Trace a result from end to end . . . 15

1. How to read this guide

This guide explains the implementation delivered in the repository, from a click in the browser to normalized quotes, matching, stake calculations, and saved scan results. It is intended for a reader who can follow basic Python and JavaScript but has not previously built an odds scanner.

Read Setup and Operations first if you want to run the program without studying its internals. Read the five separate scraper guides when configuring bookmaker pages. The scraper engine is implemented; site-specific live extraction mappings for the five requested bookmakers remain unverified until real public responses can be inspected and tested.

The project is a private, single-user application. It has no bookmaker account login, bet execution, scheduled polling, subscription system, user accounts, email service, or alert system. Those omissions follow the selected workflow: local, zero-hosting-cost, manual pre-match scans.

2. The architecture in one pass

```
Browser dashboard
  -> local FastAPI routes
  -> current settings and enabled source configurations
  -> connector registry
       -> optional provider API
       -> public HTML / JSON / anonymous browser scraper
  -> validated canonical Quote objects
  -> freshness, currency and filter checks
  -> exact event and settlement-contract grouping
  -> best eligible odds per outcome
  -> Decimal stake allocation and rounded worst-case payout
  -> opportunities and rejection reasons
  -> SQLite scan history
  -> JSON response rendered by the dashboard
```

Each stage has a distinct responsibility. A connector should not decide that two bookmakers refer to the same event; the engine does that. The engine should not parse an HTML class name; the connector does that. The frontend should not recalculate a different stake plan from binary floating-point odds; it displays the backend's monetary results.

2.1 Why Python and a small browser frontend

Python has mature HTTP, parsing, validation, browser automation, cryptography, and testing libraries. FastAPI supplies a small local JSON server. Plain HTML, CSS, and JavaScript provide the dashboard without a separate Node build, paid UI library, external CDN, or frontend hosting service.

SQLite keeps setup small and portable. It stores settings, source configurations, encrypted API credential contents, and bounded recent scan history. There is no database service to start and no external account needed for persistence.

2.2 Main files

- `pyproject.toml`: package metadata, runtime dependencies, optional browser dependency, test and PDF dependency groups, and the CLI entry point.
- `uv.lock`: exact resolved dependency versions and artifact metadata used by frozen installation.
- `arbscanner/__main__.py`: local command-line startup.
- `arbscanner/app.py`: HTTP routes, request guards, manual scan orchestration, source persistence, and PDF downloads.

- `arbscanner/models.py`: validated canonical quotes and scanner settings.
- `arbscanner/engine.py`: matching, freshness gates, market completeness, stake allocation, and opportunity output.
- `arbscanner/storage.py`: SQLite tables, settings, sources, encrypted credential storage, and recent history.
- `arbscanner/demo.py`: clearly synthetic, future-dated sample quotes.
- `arbscanner/connectors/__init__.py`: connector dispatch.
- `arbscanner/connectors/base.py`: source results, profile metadata, and source error types.
- `arbscanner/connectors/network.py`: bounded public HTTPS reads, robots handling, destination checks, and challenge detection.
- `arbscanner/connectors/parsers.py`: declarative HTML and JSON field extraction and explicit transforms.
- `arbscanner/connectors/scraper.py`: profile-aware HTTP or anonymous-browser collection.
- `arbscanner/connectors/odds_api.py`: optional, quota-bounded The Odds API requests and normalization.
- `static/index.html`, `static/app.js`, `static/style.css`: dashboard structure, interactions, and appearance.
- `tests/`: controlled checks of the implementation.
- `tools/build_docs.py`: local Markdown-to-PDF builder.
- `docs/`: editable manuals, source evidence, and generated PDFs.

3. Startup and application construction

The `arbscanner` command points to `arbscanner.__main__:main`. The command parser accepts a local port, an optional application data directory, and an optional request to open the browser. The default port is 8000. The server binds to `127.0.0.1`; there is no public-host option in the normal CLI.

```
uv run arbscanner
uv run arbscanner --port 8001
uv run arbscanner --data-dir /absolute/private/data/path
uv run arbscanner --open
```

When a data directory is supplied, startup sets `ARBSCANNER_DATA_DIR` before importing and constructing the app. `create_app()` chooses that directory or `.data` below the current working directory, constructs a `Store`, and registers the routes. With the documented command from the checkout root, that is the checkout's `.data` directory. The data directory can also be injected directly when tests build an application.

Startup reads known connector profiles and inserts missing source records in the database. Existing source settings are preserved. New profiles start disabled and unconfigured. A named profile is metadata about a source, not evidence that live extraction is ready.

Uvicorn serves FastAPI and the static dashboard. The server's access log is disabled so routine query strings are not written by that log. Application error messages also avoid returning complete upstream URLs or raw exception strings that might contain credentials.

4. The canonical Quote model

Every connector and import must eventually produce `Quote` objects. Pydantic validates them before the engine sees them. Extra fields are forbidden so accidental source-specific fields cannot silently influence matching.

4.1 Event fields

- `event_id`: a nonempty identity within the source. It is not used as a universal cross-bookmaker ID.
- `sport`: the canonical sport name used for comparison.
- `league`: the canonical competition name used for comparison.
- `home`, `away`: ordered participant names, with orientation preserved.
- `starts_at`: scheduled start as a timezone-aware timestamp.

The model requires distinct home and away names. It converts aware timestamps to UTC. A timestamp without a timezone is rejected because guessing the user's location or the source's timezone could join different fixtures.

4.2 Market fields

- `market`: one of `h2h_2way`, `h2h_3way`, `totals`, or `spreads`.
- `period`: an explicit period label, normally `full_time`.
- `rules`: a settlement label such as `regulation` or an explicitly verified overtime-inclusive contract.
- `line`: no value for moneyline; a supported half line for totals or spreads.
- `outcome`: an allowed selection for the selected market.

The allowed outcome sets are home/away for two-way moneyline, home/draw/away for three-way match result, over/under for totals, and home/away for spreads. The model rejects a draw outcome on a two-way market or an over outcome on a moneyline market.

Only half lines are accepted for totals and spreads. A total of 2.5 has no exact-equality push in an integer-scored sport under the verified scoring contract. An integer total or an Asian quarter line needs a richer settlement model, so this implementation rejects it. Unsupported data is not rounded into an allowed line.

4.3 Canonical spread orientation

For a spread market, `line` always represents the home participant's handicap. Suppose the actual selections are home -1.5 and away +1.5. Both quotes must carry the canonical `line` value `-1.5`; `outcome` identifies which selection the quote prices. The API adapter reverses an away selection's source line to obtain that home-oriented value.

A generic scraper must apply that same convention. If source rows show each selection's own sign, use an explicit transform or separate known mappings so the away value is reversed. Passing both signs through unchanged splits one market into two groups and prevents a valid comparison; forcing both to the wrong line can create an invalid comparison.

4.4 Quote and provenance fields

- `bookmaker`: the bookmaker identity, not the aggregator's identity.
- `odds`: finite decimal odds greater than 1 and no more than 10,000.

- `observed_at`: when this program obtained the observation.
- `source_updated_at`: when the source says the odds were updated, when actually available.
- `source`: connector or provenance label.
- `url`: optional HTTP(S) source link without embedded username or password.
- `currency`: a three-uppercase-letter code such as EUR.

`source_updated_at` cannot be later than `observed_at`. A connector must not invent an update timestamp to satisfy the freshness gate. The generic parsers set observation time themselves, so an untrusted remote row cannot claim a newer retrieval time than the actual fetch.

5. Settings and their units

`Settings` also rejects unknown fields and nonfinite numeric values. The initial values are a 0.5-percent minimum profit, a 100.00 total stake budget, 120-second maximum odds age, 60-second maximum quote-time spread, EUR currency, and live mode.

- `minimum_profit` is a percentage from 0 through 100. A value of 1 means one percent, not one hundred percent.
- `bankroll` is a Decimal per-opportunity budget from 1 through 1,000,000 with at most two decimal places.
- `max_age_seconds` bounds both observation age and known source age.
- `source_skew_seconds` bounds the spread between chosen legs' effective source timestamps.
- `currency` requires an uppercase three-letter code; there is no automatic foreign-exchange conversion.
- `mode` is live or demo.
- `sports_filter` and `leagues_filter` are case/whitespace-normalized substring filters.
- `require_source_timestamps` defaults true. Disabling it explicitly permits retrieval-time-only quotes with warnings.

The budget is not an account ledger. It does not know your individual bookmaker balances, minimum stakes, stake increments other than cents, maximum payouts, fees, taxes, or acceptance limits. Currency display does not imply that every currency uses a two-decimal smallest unit; this calculator uses cents consistently.

6. Connector interfaces

`fetch_source(source_id, config, api_key=None)` selects the API connector for `the_odds_api`, otherwise the public scraper connector. Keeping this dispatch small makes it easy to add a dedicated parser later without rewriting the engine.

`SourceResult` contains the source ID, status, human-readable message, validated quotes, fetch timestamp, and structured metadata. Status is one of OK, error, unconfigured, or blocked. The orchestrator uses quotes only from OK results. Failed or incomplete attempts must not silently reuse an old successful result as a new live scan.

Profiles provide IDs, names, source kind, allowed domains, supported modes, and setup notes. The configured bookmaker domains are `bet365.com`, `stake.com`, `al-1xbet.com`, `marathonbet.com`, and

`bet105.ag`; the user's Marathonbet and 1XBet pages use `/en`. These paths are part of a URL, not allowed-host entries.

6.1 Why there are no guessed bookmaker selectors

An HTML class name or private-looking endpoint found in an old example is not evidence of the current regional site. The generic scraper is executable code, but each real site's URL, selectors, field paths, status handling, and settlement interpretation require actual validation. Until then, the source reports unconfigured or blocked. The evidence ledger records that limitation.

7. Public network reads

`validate_public_url()` parses the URL, requires HTTPS on the default HTTPS port, rejects embedded credentials and fragments, checks the configured allowed hosts, and rejects local or private destinations. It resolves the hostname and requires public addresses. Redirect destinations are checked as well.

The allowed-host check permits the exact configured domain and its subdomains. That is still a source boundary: a URL on an unrelated domain must not become an arbitrary proxy request. DNS validation is a useful layer but is not a complete defense against all DNS-rebinding or local-network conditions; keep the app private and use operating-system network isolation when needed.

The HTTP client preserves the environment's proxy and certificate configuration, uses timeouts, limits redirects, and caps response bodies at five million bytes. Requests are GET-only for public collection. Host locks and last-request times enforce a delay between requests, including robots requests. A source's configured delay cannot fall below the implementation's minimum.

7.1 Robots and denial behavior

The network helper requests `robots.txt` before the page and checks the scraper's descriptive user agent. An explicit disallow blocks extraction. A missing robots file reported as 404 or 410 is treated differently from a denied or failed robots lookup. A supported crawl-delay can increase the request delay.

HTTP 401, 403, 429, and 451 become blocked-source results. Recognized challenge text also stops extraction. There is no stealth fingerprint module, CAPTCHA solver, residential proxy service, session-cookie import, login form automation, or restriction bypass.

7.2 Anonymous browser mode

Browser mode is optional. It checks the URL and robots access, starts Chromium headlessly, creates a fresh context without user cookies, blocks service workers, and routes requests through destination checks. Images, media, and fonts can be skipped because they are not needed to read odds. Non-GET requests are rejected, so a site that needs GraphQL POST or an authenticated interaction may not work in this mode.

The browser mode keeps certificate verification enabled. It uses the inherited HTTPS proxy where configured. A page can wait for an explicit CSS selector before its rendered HTML is parsed. A password field or recognized challenge stops the attempt. The browser is closed in a `finally` block even when parsing fails.

Browser rendering is more expensive and more fragile than a simple public JSON response. It is not a stronger form of permission and does not turn an inaccessible site into an available source.

8. HTML extraction

`parse_html()` uses BeautifulSoup and configured CSS selectors. An optional event selector identifies event containers; a quote selector identifies selection rows within them. Each mapping entry reads text or a named attribute from either the event container or quote row.

```
{
  "event_selector": ".fixture",
  "quote_selector": ".selection",
  "mapping": {
    "home": {"selector": ".home", "scope": "event"},
    "away": {"selector": ".away", "scope": "event"},
    "starts_at": {
      "selector": "time", "scope": "event", "attr": "datetime"
    },
    "outcome": {"attr": "data-outcome"},
    "odds": ".decimal-price"
  }
}
```

These selectors describe a controlled example structure. They are not claimed to match Bet365, Stake, 1XBet, Marathonbet, or Bet105. The configured real page must actually contain the required values.

Constants supply verified values shared by the page, such as one league, market, period, rules, or currency. Avoid broad constants on a page mixing several markets or sports. A constant of `h2h_3way` is correct only if every selected row belongs to that contract.

`inactive_selector` can reject a quote row containing a suspension marker. It is the source operator's responsibility to identify real suspended/closed markup; an odds number left in a disabled button is not necessarily an offered price.

9. JSON extraction and transforms

The JSON parser supports flat quote lists and nested event/market/outcome structures. Its small path language follows dotted object keys, numeric list indexes, and `*` wildcards. It is not executable Python or JavaScript, so a mapping cannot run arbitrary code.

For flat lists, `items_path` selects rows. For nested structures, `events_path` selects events, `markets_path` selects markets relative to each event, and `outcomes_path` selects outcomes relative to each market. A field path can use `$event`, `$market`, `$quote`, or `$root` to refer to the appropriate context.

```
{
  "mode": "json",
  "events_path": "events",
  "markets_path": "markets",
  "outcomes_path": "selections",
  "mapping": {
    "home": "$event.home",
    "away": "$event.away",
    "starts_at": "$event.starts_at",
    "market": "$market.type",
    "outcome": "$quote.side",
    "odds": "$quote.price"
  }
}
```

An object mapping may include a path, an explicit default, a `values` translation dictionary, a timestamp format of `unix` or `unix_ms`, and a numeric `multiply` transform. Translation dictionaries must

use observed source values rather than guessed numeric meanings. A source's selection code 1 is not universally home, and code 2 is not universally away.

`active_path` and `active_values` reject rows whose status is not allowed. The parser then constructs the canonical Quote; invalid rows increment a rejected-row count. It does not guess odds format, local date strings, comma decimal separators, or missing participant names. Explicit `format: american` and `format: fractional` transforms can convert verified price formats to decimal odds. For example, American +150 becomes 2.50, -200 becomes 1.50, and fractional 5/2 becomes 3.50. Invalid formats and impossible values are rejected rather than guessed.

10. The optional provider API

`odds_api.py` implements opt-in v4 reads from The Odds API. It is a separate data route from bookmaker scraping. An account key is entered through the dashboard and retrieved only by the local backend when required.

The configuration requires explicit sport keys, exact provider bookmaker keys, selected standard markets, regions, settlement rules for each sport, and a request limit. The supported provider market keys are `h2h`, `totals`, and `spreads`. There is no automatic request for every available event market or all sports.

The connector requests decimal odds and ISO timestamps. It maps the provider's event participants and market outcomes to canonical values, preserves the provider's update timestamp when present, and normalizes spread signs to the home handicap. It recognizes a small set of bookmaker aliases, but source coverage must still be confirmed in the actual provider response.

10.1 Quota handling

One request per selected sport is made up to the configured maximum. The connector records remaining/used/last request headers when available and estimates an upper request-credit bound for the chosen scope. The provider's accounting is authoritative; the estimate is not a contractual billing promise.

Authentication, denial, quota, and other request failures return a source failure without exposing the key. A partial failure discards that attempt's partial quotes instead of marking a partly failed source healthy. Supported parser rows may still be rejected for unsupported lines or missing settlement rules.

The optional sports-catalog function returns active non-outright sports from the provider. Checking a catalog is itself an external operation whose availability and accounting are controlled by the provider. No key, live account, or verified five-bookmaker coverage is bundled with this repository.

11. Freshness and quote filtering

`scan_quotes()` evaluates every validated quote against the current settings and one scan time. It rejects already-started events, future observation timestamps, stale observations, stale known source prices, unknown source age when strict mode is enabled, mismatched currency, and nonmatching sport or league filters.

Rejection reasons are counted and returned. A quote count is the number collected before these gates; it is not the number of eligible market comparisons. Read rejected counts when a source supplies data but the opportunity list remains empty.

The default source-timestamp requirement is deliberately strict. If you disable it, the effective comparison time for those quotes becomes observation time and the opportunity carries an unknown-age warning. That option is a stated limitation, not an assertion that the underlying price is fresh.

12. Event and settlement matching

`normalized()` applies Unicode NFKC normalization, case folding, and whitespace collapse. It preserves punctuation and meaningful diacritics rather than stripping identities aggressively. The group key contains normalized sport, league, ordered home and away names, exact scheduled UTC start, market type, period, rules, line, and currency.

Source-local event IDs are excluded from this cross-bookmaker key. Two bookmakers can use different event IDs for the same fixture. Conversely, a shared-looking integer ID does not establish that two unrelated sources describe the same match.

There is no fuzzy matching, alias dictionary, start-time tolerance, automatic home/away swapping, or league-name translation. If one source says Football and another Soccer, or their league labels differ, they will not match until their connector outputs are deliberately normalized consistently. This can miss real opportunities, but avoids guessing event identity.

12.1 Latest quote and conflicts

Within a matching market, the engine keeps the latest observation for each bookmaker and outcome. If equally timed rows for the same bookmaker and outcome disagree on odds, that selection is treated as conflicting and excluded. A newer observation supersedes the older one.

The engine checks that every required outcome exists, then examines coherent quote-time windows within the configured spread. Within a window it chooses the highest usable odds for each outcome and requires at least two distinct bookmakers. If the best prices all come from one bookmaker, it tries a best alternative bookmaker for an outcome rather than accepting a single-book comparison.

This is a conservative time-window comparison, not a search over arbitrary partial-settlement contracts or unlimited stake constraints. It does consider a lower-priced quote when that quote belongs to a compatible timestamp window, rather than letting an incompatible global best price hide every alternative. The engine returns one best accepted rounded plan per event/contract group, not every possible bookmaker combination.

13. Arbitrage math and stake allocation

For a complete set of mutually exclusive outcomes priced at decimal odds `oi`, the implied reciprocal sum is `S = sum(1 / oi)`. If `S >= 1`, the selected prices do not form the simple arbitrage contract modeled here. If `S < 1`, an ideal equal-return stake plan exists before rounding and real execution effects.

```
ideal_stake_i = budget / (oi * S)
ideal_payout  = budget / S
ideal_profit  = budget * (1/S - 1)
ideal_ROI%    = (1/S - 1) * 100
```

`1 - S` and `1/S - 1` are different quantities. The first is a reciprocal-probability margin; the second is the return on the total stake used by the scanner. Do not display the former as ROI.

13.1 Decimal arithmetic

Prices, lines, stakes, payouts, and budgets use `Decimal`. Decimal values avoid the binary representation surprises that can turn a boundary calculation into an accidental cent. JSON output serializes monetary values as strings so the frontend can display the intended precision.

13.2 The rounded plan

`rounded_plan()` computes proportional ideal stakes, rounds each down to a cent, and evaluates nearby cent allocations without exceeding the budget. With two or three legs, only a small finite neighborhood is needed. Zero stakes are excluded.

Each candidate's payout is stake times odds, rounded down to a cent. The smallest payout is the worst outcome. Subtracting the actual sum staked yields worst-case profit. The candidate is ranked by ROI, then profit, then total stake. The actual amount used can be slightly below the budget; the reported ROI uses the actual amount, not an assumed full budget.

Only a positive rounded profit meeting the minimum threshold becomes an opportunity. A theoretical edge that disappears after cent rounding is rejected.

13.3 Boundaries of this model

This model requires simple win/lose settlements over the complete outcome set. It does not calculate a payoff matrix for pushes, quarter lines, dead heats, partial voids, bonus bets, exchange commission, fees, taxes, different currencies, or changing execution prices. The opportunity warns that stake limits, payout caps, and actual acceptance are not modeled.

14. Manual scan orchestration

The scan route uses an `asyncio.Lock` so the same process does not run overlapping scans. A second scan attempt returns HTTP 409 rather than consuming another batch of source requests. The lock is per process; the supported CLI runs a single local application process.

Demo mode obtains dynamic synthetic quotes from `demo_quotes()` and labels the result clearly. Import mode uses the submitted validated quotes and labels them imported. Live mode reads only enabled sources. With no enabled sources, it returns an explicit warning rather than silently substituting demo prices.

Live collection uses a concurrency semaphore of two and a per-source attempt timeout. A cooldown prevents repeated manual clicks from making requests too close together. Failed sources contribute status messages and zero current quotes. The scanner then runs the engine, creates an ID and timestamps, saves the result, and returns it.

The response includes opportunities, rejection counts, collected quote count, source summaries, scan mode, warnings, and completion time. It also includes up to 100 canonical `quote_samples`, even when no opportunity passes. The dashboard's extracted quote preview lets you inspect those fields while validating a source mapping. Sorting of opportunities is by rounded ROI descending.

15. HTTP API and request guards

The API routes are local JSON operations:

- `GET /api/state`: settings, source records, key-presence boolean, and latest scan.

- `PUT /api/settings`: validate and save the complete settings object.
- `PUT /api/sources/{source_id}`: update a known profile's enabled flag and JSON configuration.
- `PUT /api/credentials/odds-api`: save an API key in its dedicated credential path.
- `DELETE /api/credentials/odds-api`: remove the saved credential record.
- `POST /api/scan`: run a manual live or demo scan.
- `POST /api/import`: validate and analyze supplied quote objects.
- `GET /api/history`: return recent saved scans.
- `POST /api/catalog/sports`: manually read the optional provider's active sports catalog using a saved key, with a cooldown.
- `GET /api/documents`: enumerate generated PDF downloads.
- `GET /documents/{name}`: serve a known PDF file by safe filename.
- `GET /openapi.json`: the local machine-readable API schema. The CDN-dependent Swagger UI is disabled.

The host guard accepts localhost identities only. The origin and browser fetch-site checks reject cross-origin or cross-site API use. Mutating JSON routes require `application/json`, and bodies are capped to avoid oversized imports or settings payloads. The server adds browser security headers and disables response caching.

The source-settings route rejects credential-looking configuration keys and credential-bearing URLs. Credentials belong in the dedicated masked API field, not the readable source JSON. Import limits and Quote validation keep malformed bulk inputs out of the engine.

These controls support a private loopback application. They do not provide user authentication or authorize public Internet deployment. A person or program with local access can still reach the application; that is why the installation should stay on a trusted personal machine.

16. SQLite and credential encryption

The store creates three tables: `kv` for settings and encrypted credential values, `sources` for source records, and `scans` for completed scan JSON. SQL parameters are bound rather than concatenated from input. A lock coordinates store access within the process.

Settings are validated when read. Source records remain JSON because their fields vary by connector mode. Scan history is bounded to the latest 100 saved scans; the normal history API returns a smaller recent window.

The first startup creates a Fernet key in `secret.key`. `set_credential()` encrypts a supplied key before putting it in SQLite; `credential()` decrypts only when the backend needs it. A wrong or lost local key raises a safe error and requires re-entering the provider key.

On POSIX systems, the directory and private files use restrictive modes. Windows requires appropriate Windows account and folder ACLs; mode 0600 is not a Windows access-control guarantee. Encrypting the database credential does not protect it from a user who can read both the database and local key file.

No raw API key is returned by `/api/state`; it returns only whether a usable saved key is present. Deleting a credential removes the current database record. Old filesystem backups may still contain older encrypted copies.

17. Frontend behavior

`index.html` supplies semantic dashboard regions, forms, buttons, settings panels, source editors, and result containers. `style.css` defines the local theme, spacing, responsive grids, visible status states, and keyboard focus styles. There is no external font or script dependency required to open the dashboard.

`app.js` loads `/api/state`, populates controls, submits settings and source changes, saves or deletes the optional key, triggers manual scans, and renders completed results. Source data is displayed as text rather than interpolated as executable HTML. Error states remain visible instead of pretending an unsuccessful scan completed normally.

The UI distinguishes demo, imported, and live results. It displays source health and odds counts so an empty result can be interpreted. A scan-in-progress state prevents accidental repeated clicks, while the backend lock and cooldown enforce the corresponding rule even for direct API calls.

The browser does not store the API key in local storage. It sends the value to the local dedicated endpoint, clears the entry field, and later displays only key-presence status. The API key belongs to the backend's provider request, not to a request made directly from the frontend to the odds vendor.

18. Tests and what they prove

The tests use controlled quotes, pages, payloads, time instants, mock network behavior, and temporary storage. Useful cases include two-way and three-way payouts, cent rounding, incomplete outcome sets, exact settlement matching, stale/future timestamps, unknown source age, mixed currencies, lines, quote conflicts, malformed fields, scraper mapping, URL restrictions, and local API behavior.

Run the suite with `uv run pytest`. Read the test runner's actual pass/fail counts. A fixture-based scraper test proves that the parser understands the fixture; it does not prove that a live bookmaker currently serves that structure. A successful HTTP mock does not prove real provider account coverage.

When adding a new source, keep a small sanitized representative fixture and include at least one suspended selection, malformed row, and unsupported line. Tests should fail if your parser turns a suspended or incomplete market into a usable complete contract.

19. A safe way to extend the program

19.1 Add a dedicated connector

Implement a source-specific fetch function returning `SourceResult`, add profile metadata, and dispatch to it in the connector registry. Keep network checks, timeouts, robots behavior, and secret handling consistent. Preserve precise source-update information and settlement identity. Add tests before enabling the source by default.

19.2 Add a market type

First define the exact outcome set and payoff contract. Extend the Quote market/outcome validation, grouping key if new contract dimensions are needed, and the engine's outcome table. For contracts with pushes or partial wins, replace the simple payout assumption with an explicit settlement matrix; do not merely add another string to the allowed market list.

Then test all settlement outcomes, line signs, period boundaries, void conditions, rounding, fees if modeled, and malformed source inputs. Update the dashboard labels and manuals. Expanding the interface's dropdown without modeling the contract does not add market support.

19.3 Add event aliases

If exact matching misses equivalent names, a future alias system should use explicit sport/league/participant-scoped mappings with auditability. A broad string-similarity join is unsafe for similarly named teams and youth or reserve events. Preserve a way to inspect and reverse a mapping rather than silently rewriting identities.

19.4 Add fees or balances

Per-bookmaker balances, minimum stakes, limits, and commission require constraints in the stake optimizer, not just extra display fields. Test that every proposed leg remains feasible and that all modeled settlement paths meet the requested net-profit threshold. Keep unmodeled costs visibly unmodeled until those tests exist.

20. PDF generation and documentation maintenance

`tools/build_docs.py` reads the seven Markdown manuals, renders headings and paragraphs, wraps source code, and creates A4 PDFs using ReportLab. It adds a cover, linked contents, outline bookmarks, headers, and page numbers. The build works without an online service or account.

```
uv sync --frozen --group docs
uv run --group docs python tools/build_docs.py
```

The builder does not execute example code blocks, test source URLs, or audit prose. After a behavior change, update the source manual, rebuild PDFs, extract text to check important commands, and inspect representative pages for clipping. Documentation can be generated successfully while still describing outdated behavior; keep the code and manuals reviewed together.

21. Trace a result from end to end

Imagine two correctly configured public sources expose the same future two-way market at decimal odds 2.10 for the opposite selections. The connectors retrieve only public data, extract participant and contract identity, set observation times, preserve known source updates, and return Quote objects.

The scan route combines successful source results. The engine rejects any stale or mismatched quotes, groups equivalent contracts, confirms both outcomes, chooses the best price per side from at least two bookmakers, and confirms acceptable timestamp spread. It calculates a reciprocal sum below one and a cent-rounded plan of approximately half the budget on each side.

The result includes the exact event and contract, each bookmaker and leg, actual stakes, payouts, worst-case profit, timestamps, and the unmodeled-execution warning. SQLite stores that scan. JavaScript renders it as one opportunity. If the user later checks a bookmaker and sees a different price, the stored result remains evidence of a prior observation, not a live guarantee or an accepted bet.